

CSL213: Data Structure and Program Design – I

Lab Assignment

Due Date : 1st November, 2023 (By 12:00 noon)

For batch - R1

Notes

1. The assignment can be done in a group of 2 students. If done in a group, both students should be from the same practical-batch.
 2. No late submission will be allowed.
 3. No copying/sharing the code across the groups. If found copied, NO marks will be awarded to the assignment for all such groups including original authors.
 4. You need to submit your source files by
 - Attaching these files in a mail by sending the mail to dspd.assignment@gmail.comPlease attach only *.c, *.h files (and other i/p text files if any). NO *.objs and *.exe to be attached.
 5. Please mark your subject line as *DSPD-1-Assignment: Your Enrollment numbers.*
-

Problem statement

You are tasked with implementing an application that allows users to manage shared expenses amongst a group of people. Each user can spend money and share expenses with others. The goal is to calculate the net amount owed or to be paid by each user in the group. You need to implement this application using the concept of array of structures.

1. *User* structure: The structure *User* must contain the following fields
 - 3-digit User ID
 - User name
 - Total amount spent by the user
2. *Expense* Structure: Define a structure called *Expense* that represents an expense. It should include:
 - 3-digit Expense ID
 - Amount of the expense
 - User who paid the expense (reference to *User* structure, e.g. User ID)
 - List of users who shared the expense (references to *User* structures). Consider maximum 4 users in each expense.

Implement the following functions for achieving the above goal:

- a. *AddUser()*: A new user should be added using this function. Every time users are added, they need to be stored in a sorted manner based on USER ID.
- b. *AddExpense()*: An expense should be added along with references to users. While adding expenses, assume a fixed number of users. Expenses should be stored in a sorted manner according to Expense ID.
- c. *AmountOwed(User_ID)*: This function should calculate the amount owed to the user. The output should contain:
 - the total amount owed (Addition of all individual amounts owed to the user)
 - Individual user names and USER IDs with the amount they owe to the given USER ID.
- d. *Amount_to_pay(USER_ID)*: This function should calculate the amount to be paid by the user. The output should contain:

- The total amount to be paid (Addition of all individual amounts that the user owes to other people)
 - Individual User names and IDs with the amount to be paid to them by the given USER ID.
- e. `User_balances()` : This function should display the final balances for each user. This function should print out the names of users and the amount they owe or are owed.
 - f. `Settle_up(User_ID_1, USER_ID_2)` : This function takes input two USER IDs. `USER_ID_1` is the payer and `USER_ID_2` is the receiver. This function should display all the amount USER 1 owes to USER 2 with the expense ID. The user should be able to select the expense to settle based on this expense ID. When function proceeds, this expense ID structure should be updated and `USER_ID_1` record from given expense should be removed.
 - g. `DeleteUser(User ID)`: This function should delete the user if and only if all balances for the User are settled. Else it should mention the balances and ask the user to settle them before deleting.
 - h. `DeleteExpense()`: This function should delete the expense record for the given expense ID. Input: Expense ID, USER ID
 Condition: Check if expense ID and User ID combination matches a recorded expense.
 If yes: output: Expense deleted.
 Else: output: Enter correct user and expense ID.

Note: For functions `AddUser()` and `AddExpense()`, assume input is unsorted.

Constraints:

- *The number of users and expenses will not exceed 1000.*
- *User IDs and Expense IDs are unique integers ranging from 1 to 1000.*
- *The amount of money spent or shared is a floating-point number with up to two decimal places.*

CSL213: Data Structure and Program Design – I

Lab Assignment

Due Date : 1st November, 2023 (By 12:00 noon)

For batch – R2

Notes

1. The assignment can be done in a group of 2 students. If done in a group, both students should be from the same practical-batch.
 2. No late submission will be allowed.
 3. No copying/sharing the code across the groups. If found copied, NO marks will be awarded to the assignment for all such groups including original authors.
 4. You need to submit your source files by
 - Attaching these files in a mail by sending the mail to dspd.assignment@gmail.com
Please attach only *.c, *.h files (and other i/p text files if any). NO *.objs and *.exe to be attached.
 5. Please mark your subject line as *DSPD-1-Assignment: Your Enrollment numbers.*
-

Problem Statement:

Define a structure having 5 members' viz Open, High, Low and Close price of stock and volume.

Note : You can visit

https://www1.nseindia.com/live_market/dynaContent/live_watch/equities_stock_watch.htm; here data is available and downloadable in csv file from above URL; OR *you can also take some dummy data if you desire.*

For each stock you can store data from *FIRST_DATE* and *LAST_DATE*. Take data of any 10 stocks for days in between *start_date* and *end_date*.

Write functions for

- i To find % change in the price of a stock from start to end date specified by the user.
- ii To sort given stock <volume-wise, close prices> wise on a particular day, which means data is sorted first on volume and for equal volume based in close prices
- iii Take a date on which a stock is bought and a date on which it is sold. Find profit/loss of two or more stocks, for these two dates.
- iv To find the highest performing stock for a specified duration, which means for a given duration (starting date and ending date), find out the stock which has highest profit / gain in terms of %age. For e.g is “s” is starting day price and “e” is ending day price, gain is $(e-s)/s * 100$
- v To find lowest performing stock for the specified duration, which means for a given duration (starting date and ending date), find out the stock which has lowest profit / gain in terms of %age. For e.g is “s” is starting day price and “e” is ending day price, gain is $(e-s)/s$ multiplied by 100

- vi List all stock as per increasing the overall performance, for the specified duration (starting-date and ending-date). Overall performance for a stock is calculated as summation of (%age change compared to previous day multiplied by volume of the stock traded on that day).
- vii To find the average value of all stock at any instant of time. Average value is calculated as weighted average of stock price and volume of the stock. Find average of average values of all stocks from start to end dates given by the user.

CSL213: Data Structure and Program Design – I

Lab Assignment

Due Date : 1st November, 2023 (By 12:00 noon)

For batch – R3

Notes

1. The assignment can be done in a group of 2 students. If done in a group, both students should be from the same practical-batch.
 2. No late submission will be allowed.
 3. No copying/sharing the code across the groups. If found copied, NO marks will be awarded to the assignment for all such groups including original authors.
 4. You need to submit your source files by
 - Attaching these files in a mail by sending the mail to dspd.assignment@gmail.com
Please attach only *.c, *.h files (and other i/p text files if any). NO *.objs and *.exe to be attached.
 5. Please mark your subject line as *DSPD-1-Assignment: Your Enrollment numbers.*
-

Problem Statement:

A city is divided into 'n' no. of different climate stations. Each station has some urban parameters and weather parameters. Urban parameters include

1. Sky view factor (range from 0.1 to 0.9)
2. Vegetation density ratio (%age value)
3. Population density (Number of people per Sq. km)
4. Pervious surface fraction (range from 0 to 15)
5. Impervious surface fraction (range from 0 to 15)
6. Station status (urban/rural)
7. Distance of station to other stations (in km)

Weather parameters are air temperature, relative humidity recorded for each station every day for one month.

Write program to provide solution to the following queries:

1. List the stations with highest heat degree days.

Heat degree days = days on which temperature is higher than average city temperature. Average city temperature is average temperature of all stations on all days.

2. For any searched station find the thermally comfortable days.

Thermally comfortable days = days for a station on which temperature difference between current station and its nearby rural station (only one i.e. any station can have only one nearest rural station) is minimum. You should calculate thermally comfortable days for urban stations only. Use distance array to find out nearby rural station.

3. List dates for all stations with decreasing vegetation density ratio and for which the temperature was more than average city temperature.
4. List the (top 'k', k-input) stations with minimum population density with highest maximum temperature. If two stations have same population density, then list those in the order of highest maximum temperature.
5. Write a function to find the heat island intensity for all urban stations.

Heat island intensity = urban station temp – rural station temp

Heat island intensity calculation: Identify all the nearby rural station for any urban station and take the average temperature difference for two stations (urban avg. temp-rural avg. temp.).

Note. If two rural stations have same distance metric, then you should consider other metric like pervious surface fraction (higher value to be preferred), to identify the nearby rural station.

CSL213: Data Structure and Program Design – I

Lab Assignment

Due Date : 1st November, 2023 (By 12:00 noon)

For batch – R4

Notes

1. The assignment can be done in a group of 2 students. If done in a group, both students should be from the same practical-batch.
 2. No late submission will be allowed.
 3. No copying/sharing the code across the groups. If found copied, NO marks will be awarded to the assignment for all such groups including original authors.
 4. You need to submit your source files by
 - Attaching these files in a mail by sending the mail to dspd.assignment@gmail.com
Please attach only *.c, *.h files (and other i/p text files if any). NO *.objs and *.exe to be attached.
 5. Please mark your subject line as *DSPD-1-Assignment: Your Enrollment numbers.*
-

Problem Statement:

We need to develop a system that allows automation of assignment of grounds to outdoor games. Automation of ground records provides feasibility in many ways for event organizers, with respect to choice of appropriate dates, cities etc.

Prepare a database of grounds, where each ground record contains the following information.

- a. Ground name
- b. City
- c. State
- d. Names of the games it can hold
- e. Dates and times available for each game
- f. Hourly rate for the ground for a specific game
- g. Date and sport name

Keep the records sorted on ground-name and city, ground-name and city together form a key in the ground database. Also, keep a list/database of records for the scheduled games on different grounds.

Perform the following operations

- a. *getFixturesinCity()* – Lists the games scheduled in the city
input: city name, time-duration (range of dates)
output: all the sports happening in the city during input time-duration at different grounds sorted according to the ground name and sport name
- b. *getFixturesSortedonDate()*
input: all records
output: list of grounds sorted according to the date and time
- c. *isAvailable()* _ gives a list of grounds available for a game on a date/time
input: game-name, date and time
output: list of grounds available on the date and time sorted according to the state and city names

- d. *UniqueSports()* – List of sports taking place on a ground which are unique (do not happen on other grounds) in the city
Input: ground name, city
Output: list of unique sports happening in the ground
- e. *FindLocationsForGameInTimeRange ()*– Find locations (grounds, cities) for a game in the given time range.
Input: Game-name, date/time-range
Output- List of grounds (with their cities) where a particular game “can be scheduled” to be played at a particular time
- f. *ClashOfMatches ()* – Determine whether there is any clash between matches on grounds given in the groundlist on a particular date for a given game
Input : List-of-grounds, date, game
Output – List all the clashing matches for a given game on a given date among the list-of-grounds input
- g. *GiveListOfGroundsHavingAtLeastKMatches ()*– List of grounds in the city having atleast K matches in total on a given day for any of the games in the list
Input : Game-list, date, city, K
Output – List of grounds in the given city having atleast K matches in total (in that city) for any of the games given in the input game-list
- h. *SortOnPriceForAGame ()*– Sorts the list of available grounds based on prices for a game on a particular date, also given the time required for the game
Input : Game, date, time
Output – List of grounds sorted based on the prices for the given game on given date and time.

CSL213: Data Structure and Program Design – I

Lab Assignment

Due Date : 1st November, 2023 (By 12:00 noon)

For batch – R5

Notes

1. The assignment can be done in a group of 2 students. If done in a group, both students should be from the same practical-batch.
 2. No late submission will be allowed.
 3. No copying/sharing the code across the groups. If found copied, NO marks will be awarded to the assignment for all such groups including original authors.
 4. You need to submit your source files by
 - Attaching these files in a mail by sending the mail to dspd.assignment@gmail.com
Please attach only *.c, *.h files (and other i/p text files if any). NO *.objs and *.exe to be attached.
 5. Please mark your subject line as *DSPD-1-Assignment: Your Enrollment numbers.*
-

Problem Statement:

Online ordering of food now-a-days is common. Managing the system is a tedious task. The goal here is to design a system which performs the task.

Create a list of restaurants in a city. Each restaurant is with attributes:

- a. Listing of generic food items and their respective prices for given quantity.
- b. Special items available at the restaurant and their prices
- c. Special Item of the day and its price.
- d. Service time required for a given quantity of every food item.

The list of restaurant records has to be in sorted order based on the key (Name and Address) i.e. the list of restaurants is primarily sorted on name-of-restaurant. If two or more restaurants are having same name, then they are sorted on their address.

Also, users are placing orders with the restaurants. Maintain a live record of orders for each restaurant. For every order, the *order time, user-name, user-phone number, user-address, food-item ordered, quantity of food items ordered and expected delivery time* are to be maintained. The delivery time can be calculated by considering service time for all the food items (make appropriate assumptions in this calculation). The list of active/live orders should be in the sorted order of first the order time and then the delivery time.

The list of restaurant records has to be in sorted order based on the key (Name and Address).

Also, maintain a database of live records of orders for each restaurant. Every order comes with order-time, food-item ordered, quantity of food items ordered and the expected delivery time. The live records should be sorted with respect to order-time and for same order- times, based on delivery time.

The following operations are to be defined so as to make the system function properly.

- a. *getMinTime()* – Given food item and quantity, outputs the name, address and delivery time of the restaurant taking minimum time to deliver the food order.
Input: food order and quantity
Output: restaurant-name, restaurant-address, time required to deliver.

- b. *getItemAvailability()* – For the given food item, outputs the list of restaurants having the food item and respective prices.
Input: food item
Output: list of restaurants and item with price at every restaurant
- c. *placeOrder()* – Places an order for the food item.
Input: list of items, quantity, user-name, user-phone-number, user-address
Output: orderID
- d. *getItemListInSortedOrder()* – For a given restaurant, outputs list of items in the sorted order of names.
Input: restaurant name
Output: list of items in restaurant in sorted order based on name
- e. *getItemListInAreaSortedOrder()* – For a given area, prints list of food-items available in that area in the sorted order of restaurants.
Input: Area
Output: list of items available items in sorted order based on name of restaurant for restaurants in the given area
- f. *getAllSpecialItemListInSortedOrder()* - Gives list of special items in a list of restaurants
Input: restaurant list
Output: list of special items in the restaurants of input restaurant-list in sorted order based on item-name and price, also print the corresponding restaurant-name and its address
- g. *getItemOfDayListInSortedOrder()* – Gives a list of “Items-of-day” in the restaurants in the given area
Input: area
Output: list of all “item of the day” in all restaurant in sorted order based on food-item-name and food-item-price
- h. *getCommonItem()* – Finds common food-item in the restaurant list
Input: restaurant list
Output: Common items present in the restaurants of the input restaurant-list given in the sorted order based on item-name. List prices of the items in different restaurants.
- i. *getAllUniqueItem()*
Input: restaurant list
Output: unique items present in each restaurant of input restaurant-list in sorted order based on item-name and price. Also print name of the restaurant and its address.